Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name	ATHARSH S U
Register Number	24011101008
Roll Number	24110049
Class	AI-DS 'A'

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

Dataset Exploration Output

```
Dataset Summary
----------------------------------------
Training images shape : (60000, 28, 28)
Training labels shape : (60000,)
Testing images shape  : (10000, 28, 28)
Testing labels shape  : (10000,)
Number of classes     : 10
Pixel value range     : [0, 255]

TensorFlow version: 2.20.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing performed:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

Shapes Before and After Preprocessing

```
Shapes BEFORE preprocessing
----------------------------------------
x_train: (60000, 28, 28), dtype=uint8
x_test : (10000, 28, 28), dtype=uint8
y_train: (60000,), dtype=uint8
y_test : (10000,), dtype=uint8

Shapes AFTER preprocessing
----------------------------------------
x_train: (60000, 784), dtype=float32, range=[0.0, 1.0]
x_test : (10000, 784), dtype=float32, range=[0.0, 1.0]
y_train: (60000, 10), dtype=float32
y_test : (10000, 10), dtype=float32
```

```
Example: flatten of a 2x2 block [[10,20],[30,40]] -> [10,20,30,40]
[10 20 30 40]
```

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Model Summary:

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Task 4: Model Training

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

```
Epoch 1/20
  73/1688  3s 2ms/step - accuracy: 0.5004 - loss: 1.4556
I0000 00:00:1784737546.985625      81 device_compiler.h:196] Compiled cluster using XLA!  This line is logged at
     most once for the lifetime of the process.
1688/1688  8s 3ms/step - accuracy: 0.8183 - loss: 0.5035 - val_accuracy: 0.8503 - val_loss: 0.4053
Epoch 2/20
1688/1688  4s 2ms/step - accuracy: 0.8637 - loss: 0.3737 - val_accuracy: 0.8635 - val_loss: 0.3742
Epoch 3/20
1688/1688  4s 2ms/step - accuracy: 0.8770 - loss: 0.3349 - val_accuracy: 0.8623 - val_loss: 0.3754
Epoch 4/20
1688/1688  4s 2ms/step - accuracy: 0.8852 - loss: 0.3090 - val_accuracy: 0.8695 - val_loss: 0.3649
Epoch 5/20
1688/1688  4s 2ms/step - accuracy: 0.8922 - loss: 0.2892 - val_accuracy: 0.8698 - val_loss: 0.3679
Epoch 6/20
1688/1688  4s 2ms/step - accuracy: 0.8979 - loss: 0.2741 - val_accuracy: 0.8727 - val_loss: 0.3667
Epoch 7/20
1688/1688  4s 2ms/step - accuracy: 0.9025 - loss: 0.2603 - val_accuracy: 0.8692 - val_loss: 0.3768
Epoch 8/20
```

```
1688/1688  4s 2ms/step - accuracy: 0.9058 - loss: 0.2499 - val_accuracy: 0.8767 - val_loss: 0.3628
Epoch 9/20
1688/1688  4s 2ms/step - accuracy: 0.9103 - loss: 0.2384 - val_accuracy: 0.8783 - val_loss: 0.3542
Epoch 10/20
1688/1688  4s 2ms/step - accuracy: 0.9139 - loss: 0.2271 - val_accuracy: 0.8803 - val_loss: 0.3605
Epoch 11/20
1688/1688  4s 2ms/step - accuracy: 0.9169 - loss: 0.2195 - val_accuracy: 0.8802 - val_loss: 0.3588
Epoch 12/20
1688/1688  4s 2ms/step - accuracy: 0.9200 - loss: 0.2103 - val_accuracy: 0.8843 - val_loss: 0.3679
Epoch 13/20
1688/1688  4s 2ms/step - accuracy: 0.9225 - loss: 0.2034 - val_accuracy: 0.8808 - val_loss: 0.3820
...
Epoch 20/20
1688/1688  4s 2ms/step - accuracy: 0.9373 - loss: 0.1640 - val_accuracy: 0.8828 - val_loss: 0.4168

Training completed in 84.9 seconds
```

Task 5: Model Evaluation

```
Baseline Model  Test Set Metrics
----------------------------------------
Accuracy : 0.8790
Precision: 0.8795
Recall   : 0.8790
F1-score : 0.8777

Classification Report
----------------------------------------
              precision    recall  f1-score   support

 T-shirt/top       0.78      0.88      0.83      1000
     Trouser       0.99      0.96      0.98      1000
    Pullover       0.83      0.78      0.80      1000
       Dress       0.83      0.92      0.87      1000
        Coat       0.78      0.82      0.80      1000
      Sandal       0.96      0.96      0.96      1000
       Shirt       0.76      0.61      0.68      1000
     Sneaker       0.92      0.97      0.94      1000
         Bag       0.98      0.97      0.97      1000
  Ankle boot       0.97      0.92      0.95      1000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.88      0.88      0.88     10000
```

7. Source Code

Repository Link: `https://github.com/Atharsh2910/LAB2-MLP`

8. Hyperparameter Optimization

Hyperparameter optimization was performed using **RandomizedSearchCV** together with the SciKeras `KerasClassifier` wrapper, using 5-fold cross-validation.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

1. Built a baseline MLP model (Section 6, Task 3).
2. Defined the hyperparameter search space (table above).
3. Performed Randomized Search with 5-fold cross-validation.
4. Recorded the best hyperparameter combination (above).
5. Retrained the model using the optimized hyperparameters.
6. Evaluated the optimized model on the testing dataset.
7. Compared the optimized model with the baseline model.

Retraining with Optimized Hyperparameters

Optimized Model Summary:

```
Layer (type)                 Output Shape              Param #

dense_290 (Dense)            (None, 256)               200,960

dropout_176 (Dropout)        (None, 256)                     0

dense_291 (Dense)            (None, 10)                  2,570
```

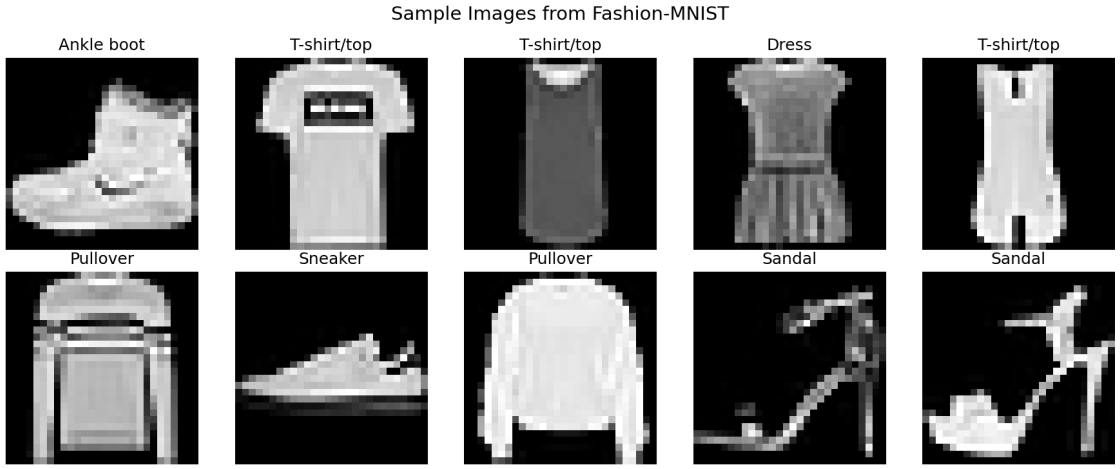
Optimized Model – Test Set Evaluation

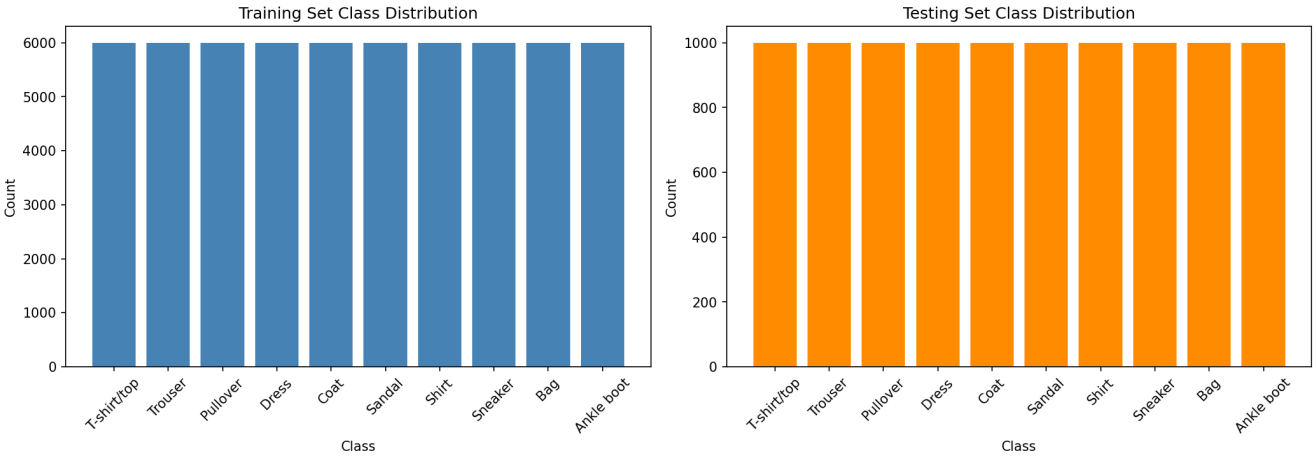
```
Optimized Model  Test Set Metrics
----------------------------------------
Accuracy : 0.8799
Precision: 0.8819
Recall   : 0.8799
F1-score : 0.8775

Classification Report
----------------------------------------
              precision    recall  f1-score   support

 T-shirt/top       0.78      0.89      0.83      1000
     Trouser       1.00      0.97      0.98      1000
    Pullover       0.74      0.83      0.79      1000
       Dress       0.83      0.92      0.87      1000
        Coat       0.79      0.78      0.79      1000
      Sandal       0.95      0.99      0.97      1000
       Shirt       0.81      0.55      0.66      1000
     Sneaker       0.95      0.95      0.95      1000
         Bag       0.99      0.96      0.98      1000
  Ankle boot       0.98      0.94      0.96      1000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
```

```
weighted avg       0.88      0.88      0.88     10000
```

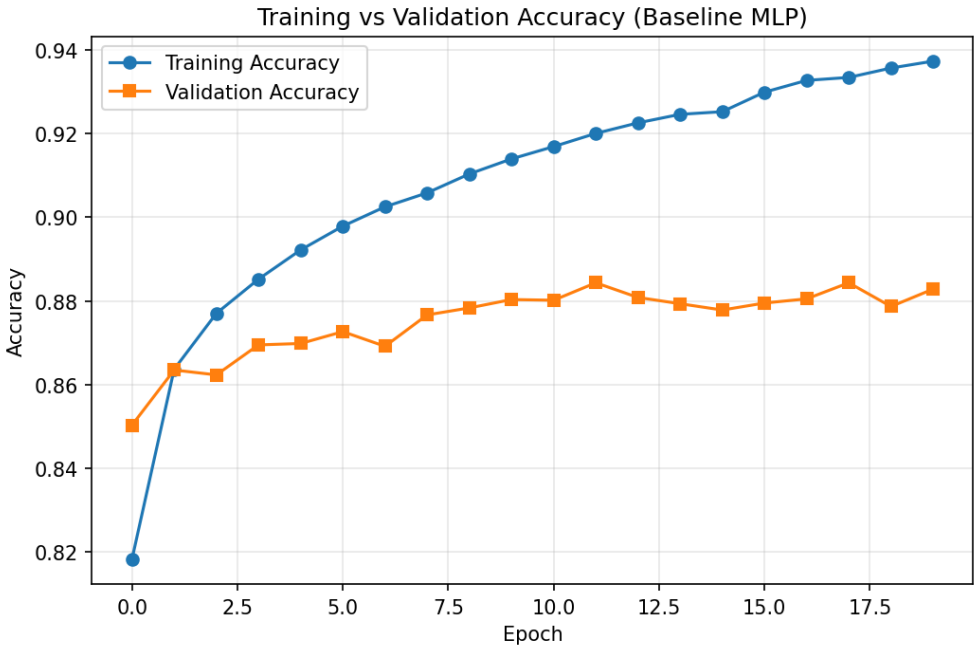
```
Accuracy improvement from hyperparameter optimization:
Baseline Accuracy : 0.8790
Optimized Accuracy: 0.8799
```

9. Mandatory Plots

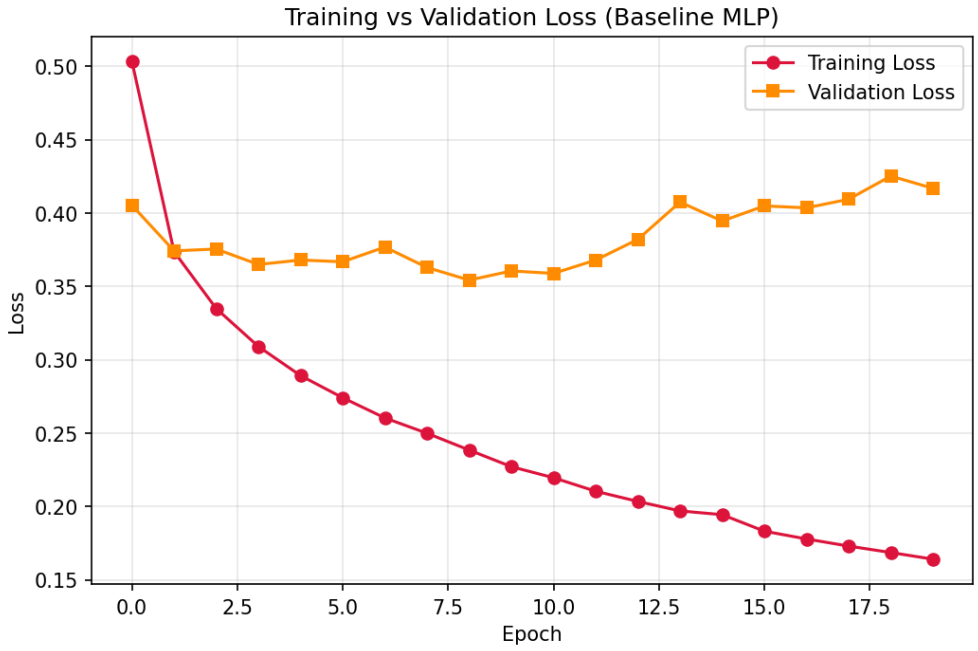




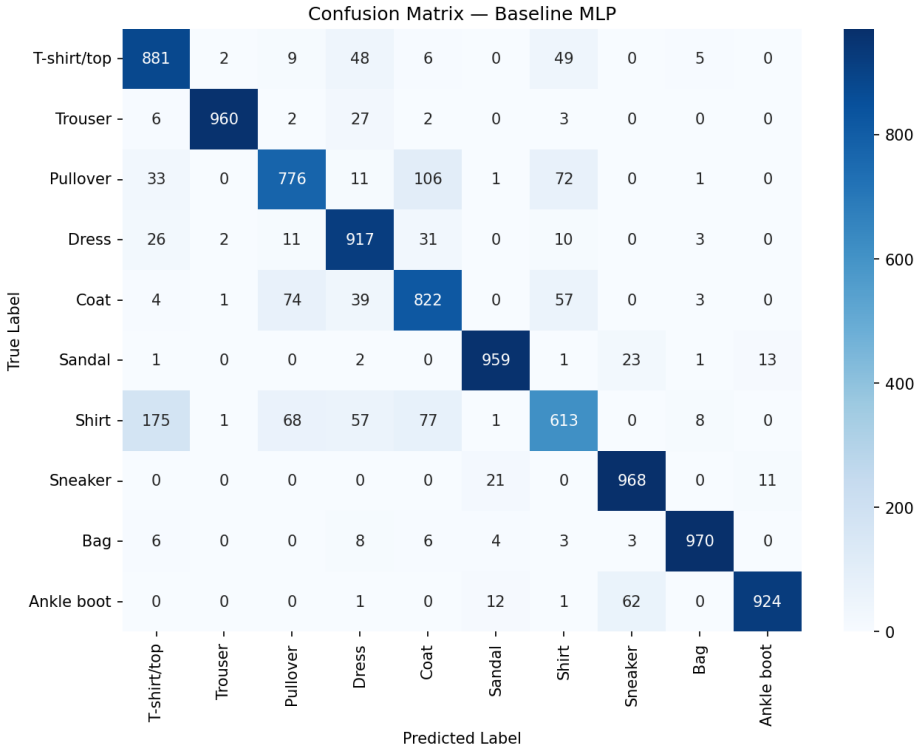
*Inference:*The graph shows that the training and test set has a normal class distribution, showing that the classes are perfectly balanced.



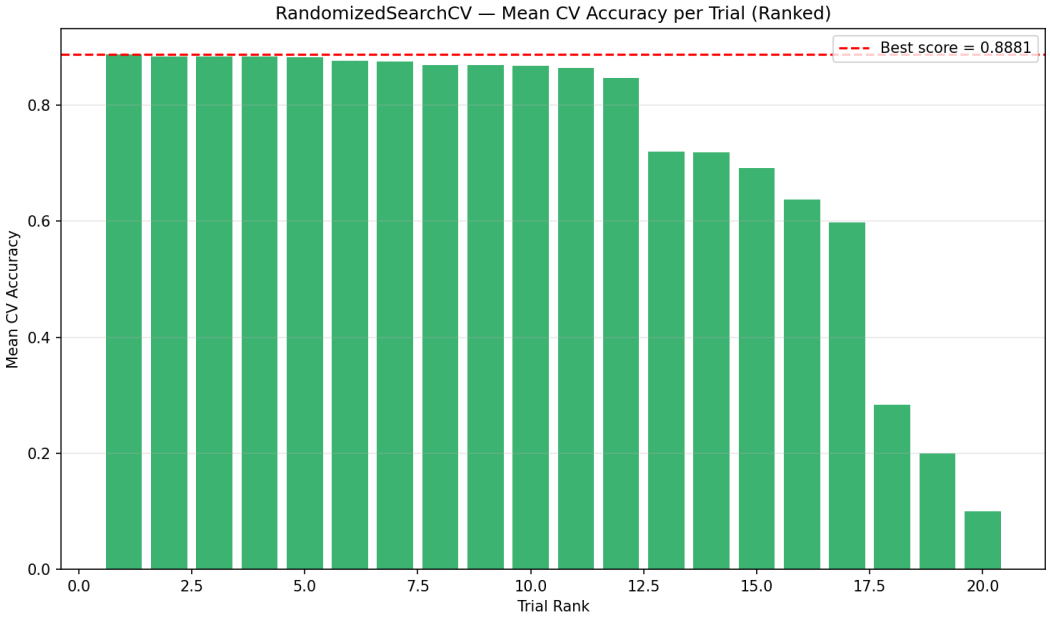
Inference: The training accuracy steadily increases across epochs, showing that the model is learning. The validation accuracy hovers around 88% showing that the model performs well in validation set, with slight overfitting.



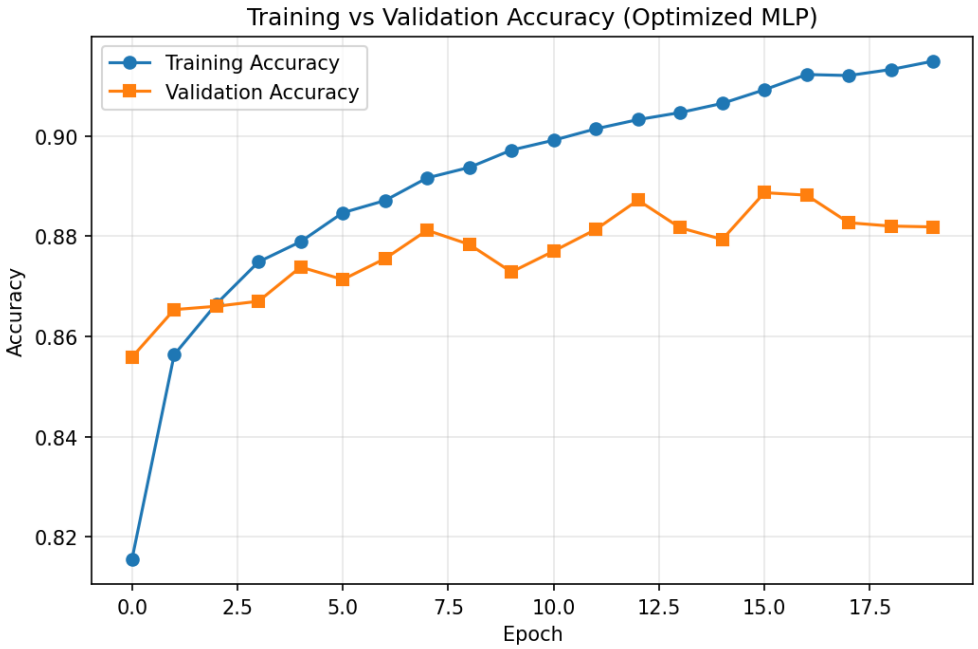
Inference: The training loss decreases steadily across epoch while the validation loss slightly increases with increase in epoch, proving slight overfitting.



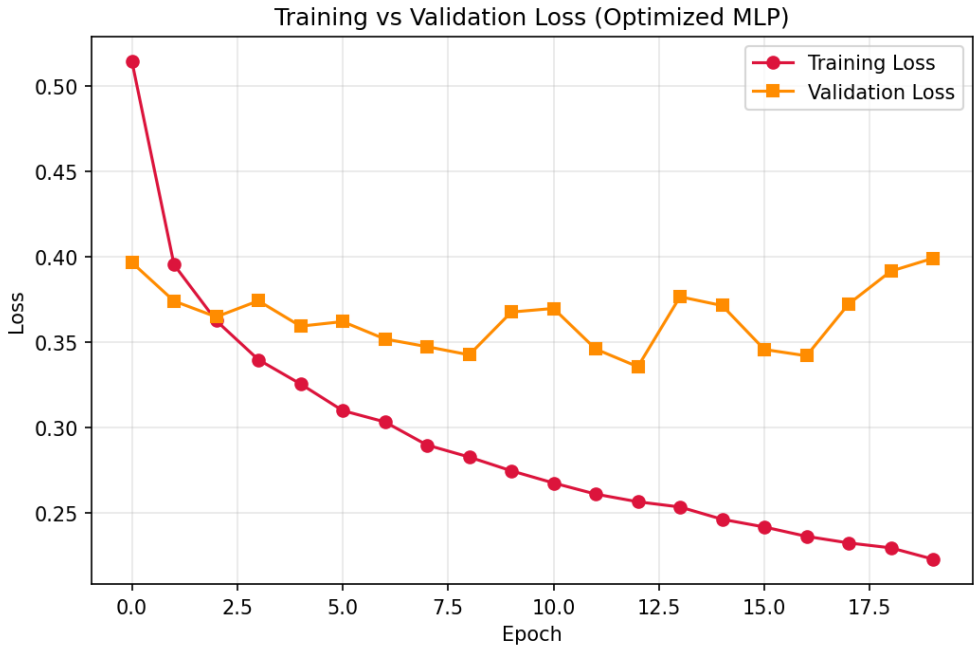
Inference: Most of the classes are classified correctly, showing that a MLP can be used for classification of the dataset, with a decent accuracy of 0.8.



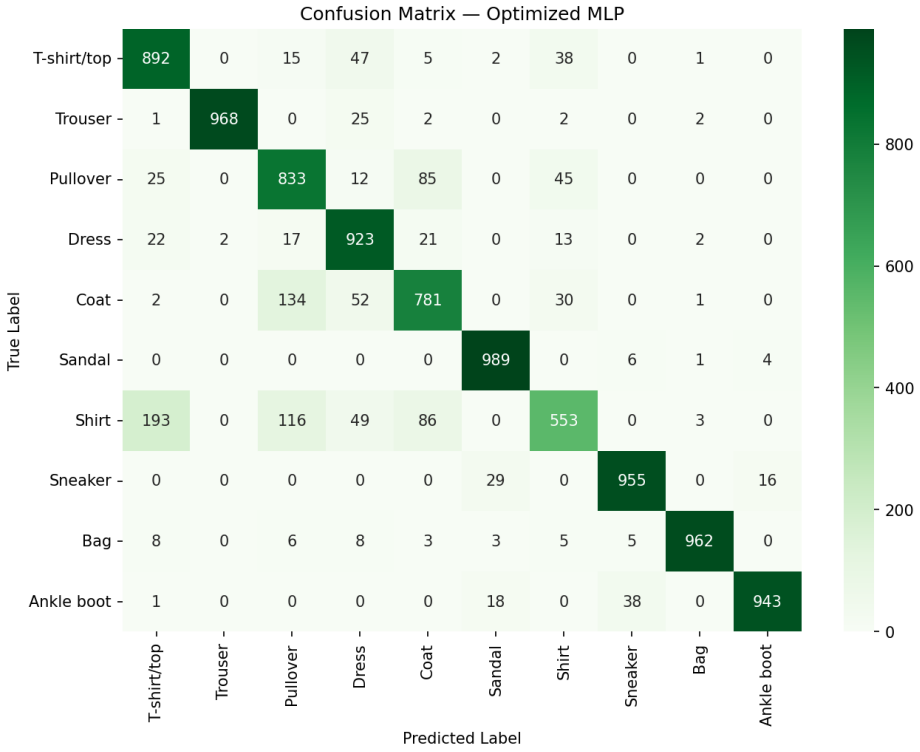
Inference: Using randomizedSearchCV, a mean CV accuracy 88% is achieved, showing that several top-ranked hyperparameter combinations achieve high performance, indicating model stability. Lower-ranked trials have reduced accuracy showing that hyperparameter selection impacts model accuracy.



Inference: The optimized MLP shows a steady increase in training accuracy, and validation accuracy hovering at 88%, indicating the optimized MLP has good learning. There is no significant difference between baseline and optimized model.



Inference: The optimized MLP exhibits a steady decrease in training loss, while the validation loss initially decreases and then fluctuates around 0.34–0.40. The slight increase in validation loss during later epochs suggests mild overfitting.



Inference: The confusion matrix shows that the optimized MLP correctly classifies most samples. Most misclassifications arise for classes that are similar.

Inference: Both the baseline model and optimized model has highly similar performance metrics, showing that MLP can achieve a maximum accuracy around 88% despite maximum optimization. There is a limit for MLP in image classification thereby we have to use CNN for better accuracy.

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	ADAM
Activation Function	RELU
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8801
Testing Accuracy	0.8799

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8790	0.8799
Precision	0.8795	0.8818
Recall	0.8790	0.8799
F1-score	0.8776	0.8775
Training Time	84.5 s (20 epochs)	158.6(20 epochs)

11. Discussion

1. **Which optimization method was used?**
   RandomizedSearchCV with 5-fold cross-validation was used, sampling 20 random configurations from the 60k samples.

2. **Which hyperparameters were selected?**
   1 hidden layer with 256 neurons, ReLU activation, ADAM optimizer with learning rate 0.001, dropout rate 0.2, batch size 16, trained for 20 epochs. This combination achieved the best mean cross-validation accuracy of 0.8799.

3. **Did optimization improve performance?**
   No. The optimization did not create any improvement in the model performance. Both baseline and optimized had almost similar validation accuracy. This is because of the limitation of MLP on image dataset. A 28*28 dataset creates 784 input neurons, 784 hidden neurons and 784 bias. This huge size of the perceptron leads to slower learning, thereby affecting the accuracy. Though the model gave an accuracy of 88%, there was no improvement upon optimization due to the structure of the MLP.

4. **Which hyperparameter had the greatest impact?**
   ReLU activation function, with ADAM Optimizer with 1 hidden layer, 256 hidden neuron and lr of 0.001 had the greatest impact. It gave the highest accuracy with fastest convergence.

5. **Compare Grid Search and Randomized Search.**

Grid Search evaluates every combination in the search space, whereas randomsearch evaluates a fixed number of random samples. Grid search gives the highest accuracy but random search is used for larger search space.

6. **Recommend the best MLP configuration.**
   Activation Function - ReLU
   Optimizer - ADAM
   Hidden layers - 1
   Hidden neuron - 256
   LR - 0.001
   Batch size - 16
   Dropout rate - 0.2

12. Conclusion

The MNIST Dataset was trained and validated using a multi layer perceptron. The baseline model was built using 784 input neurons, ReLU & Softmax activation function, without optimizers. It produced an accuracy of 0.8790. The model was further optimized using various hyperparameters, and the best combination produced an accuracy of 0.8799. Both baseline and optimized version produced almost similar accuracy, proving the limitations of MLP on image dataset, thereby leading to the use of CNN for image dataset.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.